

Implementation of a Basic Blockchain with PoW and Merkle Patricia Tries

Joel González Jiménez
Universitat Politècnica de Catalunya – FIB
Barcelona, Spain

Abstract—Blockchain research is continually driving new functionalities and optimizations. As a result, modern blockchains look vastly different from the beginning. This paper presents a modular, localhost blockchain prototype written in Go, specifically engineered to study the core concepts such as consensus mechanisms, Proof-of-Work (PoW), and cryptographic proofs with modified hexary Merkle Patricia Trie (MPT). By decoupling these pillars from the overhead of peer-to-peer networking, we evaluate structural state integrity, deterministic block-hashing, and disk-bound persistence via an embedded key-value data-store layer. Crucially, we leverage some optimizations and systemic constraints of a MPT implementation that utilizes self-referential content-addressed hashing to achieve more efficient space usage.

Index Terms—Blockchain, Merkle Patricia Trie, Proof of Work, Cryptographic Proof.

I. INTRODUCTION

Blockchain research is continually driving new functionalities and optimizations. As a result, modern blockchains look vastly different from the beginning. While early distributed networks relied strictly on simple transaction sequences appended to linear ledgers, contemporary platforms require fine-grained state tracking and highly optimized authenticated storage configurations.

This project is a hands-on implementation of an experimental blockchain environment designed to deliberately isolate and study these core foundational subsystems, focusing comprehensively on:

- **Consensus Mechanism (PoW):** The throttling of ledger updates through intensive algorithmic resource consumption.
- **Merkle Patricia Trie (MPT):** The design of an authenticated, radix-based data organization layer capable of emitting cryptographic proofs.

II. CORE SYSTEM CAPABILITIES

The developed platform provides a robust research testbed that successfully demonstrates four primary system capabilities:

- **Cryptographic Immutability:** Enforcing absolute historical integrity across the ledger via sequential SHA-256 block-hashing tied directly to the execution bounds of the Proof-of-Work consensus loop.
- **Lightweight Verification:** Utilizing a custom Merkle Patricia Trie layout to yield deterministic cryptographic

execution proofs, maintaining an efficient operational lookup bounds of $\mathcal{O}(\log n)$.

- **Persistent State Storage:** Implementing a reliable underlying data-store layer utilizing an embedded B+ tree database to preserve the entire blockchain structure across successive experimental runs.
- **Debugging Tools and Tests:** Integrating diagnostic utilities and localized test suites optimized for analyzing the serialization size, runtime growth, and data density of individual block payloads.

III. CUSTOM MERKLE PATRICIA TRIE DESIGN

The data storage architecture modifies the traditional execution parameters of a standard Merkle Patricia Trie to optimize for a per-block authenticated data log environment.

A. Architectural Departures from Standard MPT

Unlike generalized global world-state implementations (such as Ethereum), this framework enforces four explicit structural constraints:

- 1) **GOB Encoding Framework:** The system substitutes standard Recursive Length Prefix (RLP) serialization for Go's native GOB binary encoder to natively streamline abstract interface polymorphism across node variations.
- 2) **Value-Free Branch Nodes:** Standard MPT branch nodes store a terminating value slot to accommodate variable-length paths. In this framework, **Branch nodes do not contain a value** field because every key in the sub-tree shares a strictly uniform static length constraint.
- 3) **Self-Referential Content Addressing:** The path/key utilized to navigate to a value inside the trie is uniquely derived as the SHA-256 hash of the target value itself:

$$\text{Path} = \text{SHA256}(\text{Value})$$

Consequently, every key mapping to a node spans exactly 32 bytes (64 hexary nibbles), eliminating path variance.

B. Node Structural Layouts

An example of the structure implemented Merkle Patricia Trie can be seen in Figure 1. There are three different configurations of a Node:

- **Leaf Node:** Contains the terminal path slice directly to its corresponding raw data payload.

- **Branch Node:** Is an array of 16 slot hashes (Childs_hash [16][32]byte) to manage routing through nibbles.
- **Extension Node:** Leverages path compression by squashing a sequence of shared nibbles (Path_shared) and contains a reference of the next hash (Next_hash).

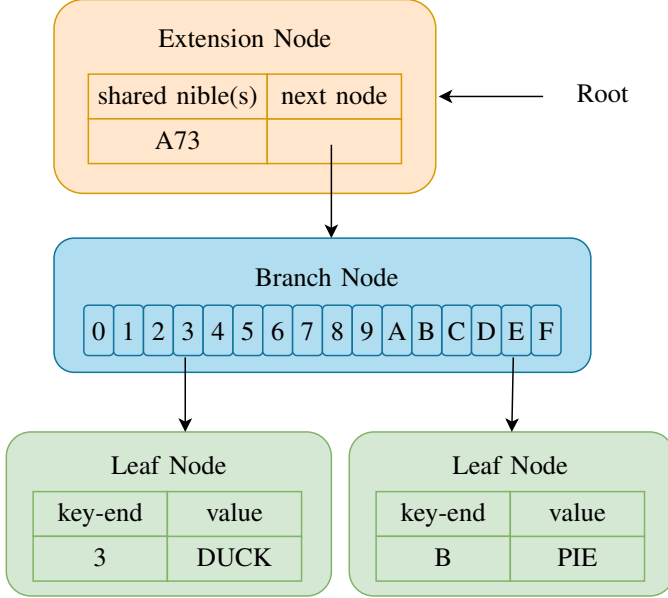


Fig. 1. Custom hexary Merkle Patricia Trie

IV. SERIALIZATION ANALYSIS: GOB VS. RLP

A foundational design trait of this prototype is its reliance on Go's internal binary encoder (GOB) instead of the Recursive Length Prefix (RLP) specification found in modern public blockchains like Ethereum.

A. The GOB Rationale

GOB provides exceptional utility inside a specialized, single-language Go prototype. It inherently processes abstract interface schemas. By embedding type descriptions and meta-data definitions directly inside the resulting byte array, it allows the storage layer to seamlessly serialize and deserialize abstract Node definitions without requiring manual byte-parsing and validation boilerplates.

B. The RLP Consensus Imperative

While optimal for a localized framework, GOB is critically flawed in distributed multi-node consensus environments because it lacks absolute binary determinism. Internal stream optimizations and compiler variant updates can yield subtle discrepancies in output byte arrays from identical input data, causing irreversible ledger forks. Implementing **RLP** represents a vital optimization upgrade for production deployment; RLP omits all struct type schema data, encoding exclusively lengths and raw bytes to ensure absolute cross-client cryptographic determinism.

V. CONSENSUS ENGINE: PROOF-OF-WORK

To commit an assembled data block securely to the persistent ledger, the mining module (`pow.go`) forces computational verification.

A. Target Boundary Evaluation

A block's validity is governed by a strict mathematical inequality constraint. The system boundary target big integer T is calculated by executing a left-shift operation against a baseline value based on a globally declared block difficulty variable D :

$$T = 1 \ll (\text{HashLength} - \text{Difficulty}) \quad (1)$$

Where HashLength represents the bit-width of the cryptographic hashing output space (256 bits).

B. The Execution Loop

The consensus engine loops through individual incrementing iterations of a block's 64-bit integer `Nonce` field, validating that the resulting double-pass hash satisfies the inequality:

$$\text{SHA256}(\text{BlockData} \parallel \text{Nonce}) < T \quad (2)$$

Once a nonce is discovered that positions the integer representation of the block hash below the calculated target line T , the mining cycle terminates, and the block is broadcast to the storage management layer.

VI. SYSTEM ARCHITECTURE AND CLASS DIAGRAM

The architectural design strictly prioritizes modularity, enforcing a highly structured, acyclic dependency graph among the application classes to clearly delineate operational responsibilities.

- **The Blockchain Class:** Acts as the primary entry point, maintaining a direct strong composition relationship over the active `Trie` model and overseeing database transactions.
- **The Storage Management Layer:** Utilizes an instance of `boltStorage` to write data into two buckets ("blocks" and "tries").
- **The Iteration Layer:** The `BlockchainIterator` manages a weak aggregation relationship back to the database, tracking byte-serialized headers backwards via the block's embedded `PrevBlockHash` hash pointer.

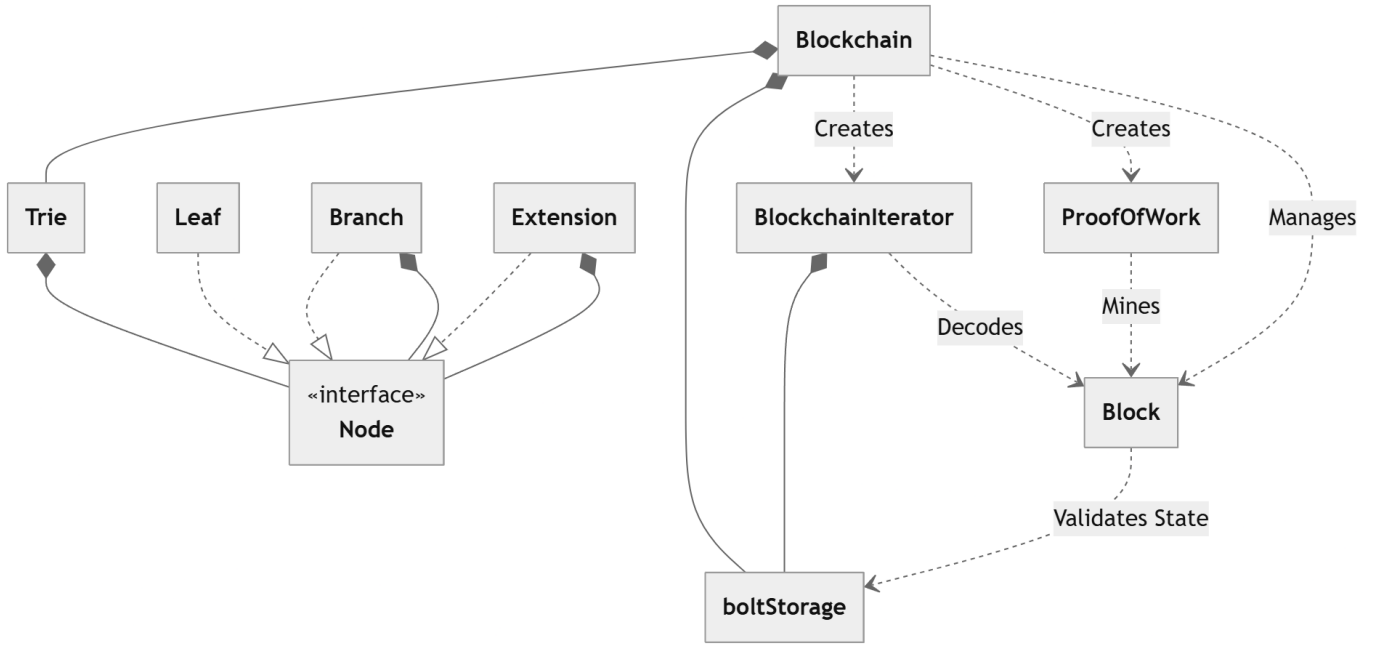


Fig. 2. Class diagram

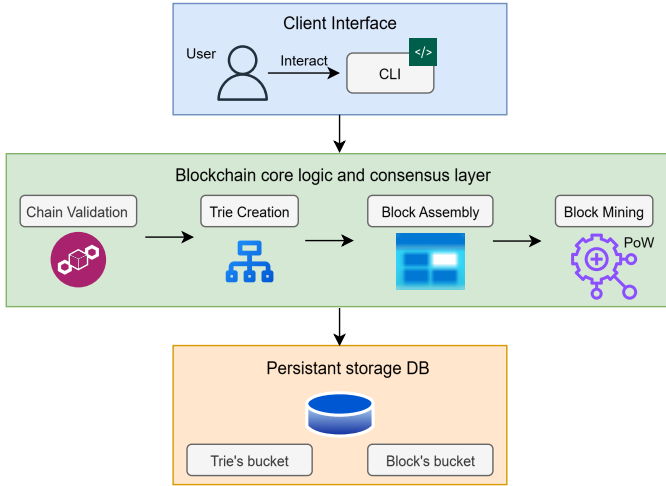


Fig. 3. Architecture

VII. CRITICAL TRADE-OFFS AND VULNERABILITIES

A. Vulnerability Assessment

- Lack of Authorization and Signature Verification:** Payloads are processed as arbitrary data strings. The missing implementation of public-key cryptography (such as ECDSA or Ed25519 digital signatures) creates a vulnerability where transaction ownership or authenticity cannot be validated.
- Exposure to State Bloat DoS Attacks:** The absence of an economic gas or processing fee model allows a malicious agent to execute infinite value insertions for free, risking host disk space exhaustion.

VIII. CONCLUSION AND FUTURE WORK

This paper successfully presented a modular, clean research framework implemented in Go that successfully isolates and studies Proof-of-Work constraints and specialized Merkle Patricia Trie behaviors. By mapping paths strictly via content hashes, the MPT architecture successfully optimizes out branch value requirements. Future milestones will focus on transitioning this block-centric log structure into a cumulative, rolling world-state engine by persisting root hash variables across block headers, replacing the GOB encoder with a deterministic RLP framework, and establishing cryptographic signature enforcement mechanisms.